
CNS6214 Operating Systems

Distributed Producer-Consumer Transfer & Parallel Analytics

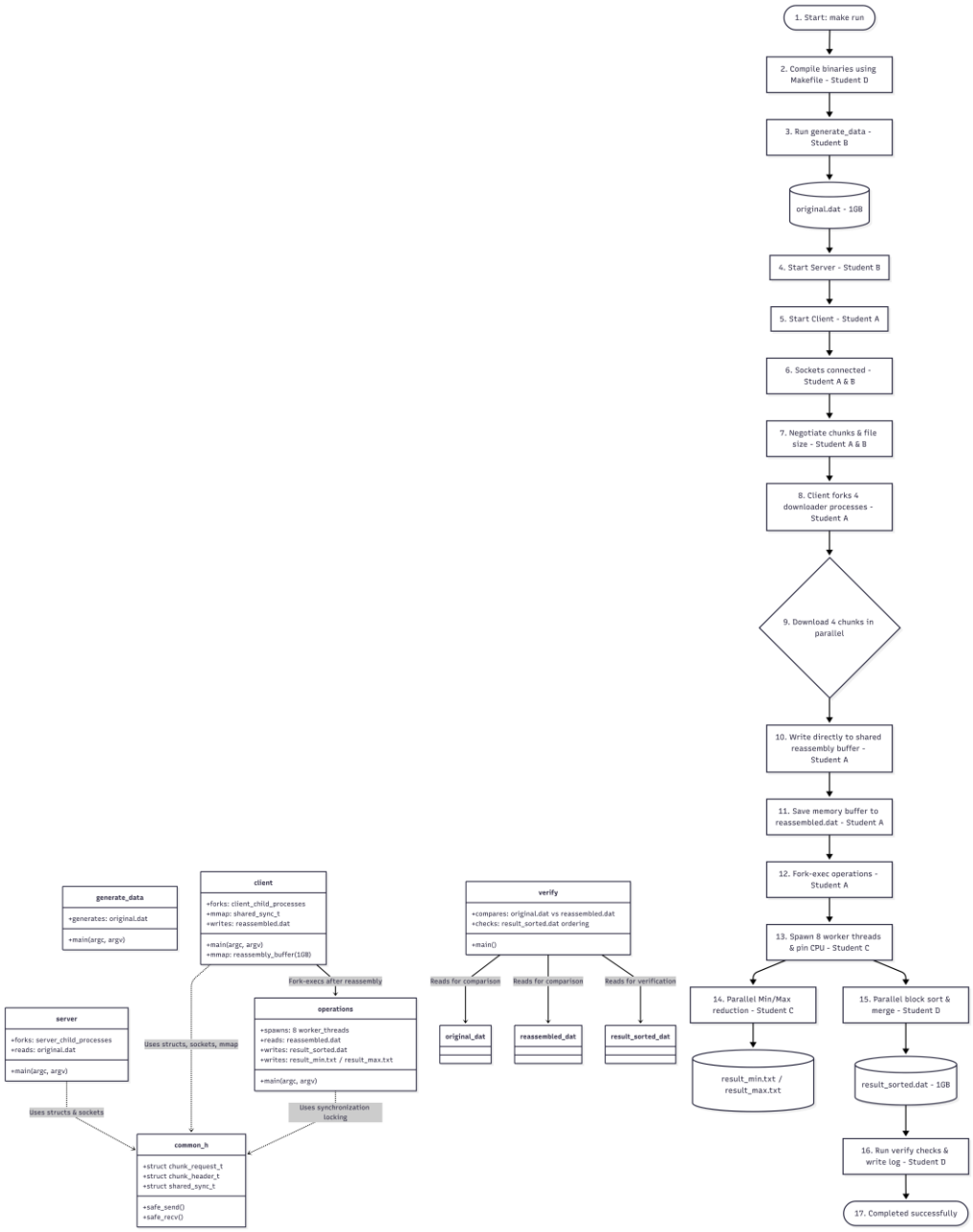
Assignment (20%)

Prepared by:


| No | Name | Student ID |
|---|---|---|
| A | ALHALAH KUTAIBA | 242UC241CY |
| B | MUHAMMAD AMMAR BIN MOHAMAD IDHAM | 261UC260P2 |
| C | MUHAMMAD NAFIZ BIN MOHD FAUZI | 253UC255LY |
| D | MUAZ WAZIF BIN ABD MANAB | 261UC260XS |


GitHub link: https://github.com/muazwzif/GroupX_OS_Project.git

## Architecture Diagram and Flowchart



## Synchronisation Strategy :

The system is synchronized through 2 layers, the process level sync for chunk reassembly after the client receives the data generated file, the 2nd layer is threads syncing together to process the data file received, they analyse it, and sort it accordingly

1-layer one of processing :

Because child processes operate in a different virtual address space, the heap memory allocation “malloc” cannot share data , to coordinate the processes to reassemble the data file we needed :

1.1 shared memory: we mapped the 1gb shared data memory using “mmap()” with mapshared

|mapannonymouse to hold the reassembled along a control structure “shared_sync_t”

Client.c L 120

```
// Map shared synchronization controller
shared_sync_t *sync_ctrl = mmap(NULL, sizeof(shared_sync_t), PROT_READ | PROT_WRITE,
                                MAP_SHARED | MAP_ANONYMOUS, -1, 0);
if (sync_ctrl == MAP_FAILED) {
    perror("mmap shared sync controller failed");
    munmap(reassembly_buffer, file_size);
    for (uint32_t i = 0; i < N; i++) close(client_fds[i]);
    free(client_fds);
    return 1;
}
```

1.2 process shared mutexes to avoid race conditions , intializing the PTHREAD _ PROCESS _ SHARED attribute , guards the shared chunks_received

Client.c L131

```
// Initialize synchronization primitives as process-shared
pthread_mutexattr_t mattr;
pthread_mutexattr_init(&mattr);
pthread_mutexattr_setpshared(&mattr, PTHREAD_PROCESS_SHARED);
pthread_mutex_init(&sync_ctrl->buffer_lock, &mattr);
pthread_mutexattr_destroy(&mattr);

pthread_condattr_t cattr;
pthread_condattr_init(&cattr);
pthread_condattr_setpshared(&cattr, PTHREAD_PROCESS_SHARED);
pthread_cond_init(&sync_ctrl->chunk_arrived, &cattr);
pthread_condattr_destroy(&cattr);

sem_init(&sync_ctrl->received_count, 1, 0);
sync_ctrl->chunks_received = 0;
```

1.3 process shared conditions variable : pthread_ cond *t* wakes the parent when child is completed , the parent blocks pthread cond wait to sleep efficiently :

```
// Coordinate using condition variables and semaphores
pthread_mutex_lock(&sync_ctrl->buffer_lock);
while (sync_ctrl->chunks_received < N && !child_failed) {
    pthread_cond_wait(&sync_ctrl->chunk_arrived, &sync_ctrl->buffer_lock);
}
pthread_mutex_unlock(&sync_ctrl->buffer_lock);
```

1.4 counting semaphore (sem_ t ) : initialized with (sem init) used by child process to post completion and the parents consumes it

In client.c L144

```
    pthread_condattr_destroy(&cattr);
Ctrl+L for Agent
    sem_init(&sync_ctrl->received_count, 1, 0);
    sync_ctrl->chunks_received = 0;
```

In client L243

```
+L for Agent
    sem_post(&sync_ctrl->received_count);
    exit(0);
```

Client 275

```
// Wait N times on semaphores to demonstrate counting logic
for (uint32_t i = 0; i < N; i++) {
    sem_wait(&sync_ctrl->received_count);
}
```

Layer 2: thread level sync

To guarantee thread safety throughout the multi-threaded data analytics phase, POSIX synchronisation primitives were employed to avert race problems during simultaneous data access.

2.1 a mutex lock was implemented to safeguard the global_min and global_max accumulators from concurrent write conflicts. Since multiple threads assess their local minimum and maximum values at the same time, writing to the global variables without a lock would lead to significant race conditions, resulting in corrupted final values. The lock guarantees strict mutual exclusion.

```
pthread_mutex_lock(&reduction_lock);
if (local_min < global_min) global_min = local_min;
if (local_max > global_max) global_max = local_max;
```

2.2 a condition variable was used to synchronize the completion of the local reductions. The primary execution thread is halted using pthread_cond_wait() until a shared threads_completed counter equals the total number of spawned threads. This ensures that the final outputs are produced only after all threads have completed their tasks.

```
pthread_mutex_lock(&reduction_lock);
while (completed_reductions < T) {
    pthread_cond_wait(&reduction_cond, &reduction_lock);
}
pthread_mutex_unlock(&reduction_lock);
```

## Parallelism Implementation

Two types of parallelism are implemented in this program which is data parallelism and task parallelism. To describe these concepts briefly, data parallelism involves the same task performing on different pieces of data, whereas task parallelism involves distributing different tasks across multiple cores.

**Data parallelism implementations:**

1. **File downloading** (in client.c, using `fork()`)
   The program partitions the total file size evenly among N child processes. The chunk size is calculated as `(file_size + N -1)/N`. Each child process i is assigned a specific chunk sequence number (i+1) and independently connects to the server to download it. Instead of writing to separate files, all processes write concurrently to specific offsets (written as `offset = (seq - 1) * chunk_size` in the code) within a single, shared mmap buffer.

2. **Min/Max Parallel Reduction** (in operations.c, using `pthread`)
   The global integer array is partitioned across T threads. The base chunk size is determined by `total_elements / T`. If there is a remainder, the first few threads take one extra element each to ensure all elements are covered. Each thread independently iterates over its assigned bounds to find its local_min and local_max. After the local scan, threads synchronise using a mutex to compare and update the global_min and global_max variables.

3. **Block Sorting** (in operations.c, using `pthread`)
   It reuses the same data partitions from the min/max phase. Each thread independently runs `qsort()` on its local chunk of the array. A semaphore, `active_sort_sem`, limits the maximum number of concurrent sorts to T/2 or 1 if T=1 to manage CPU cache pressure. Once a thread finishes sorting its block, it flags its segment as ready for the first level of merging.

**Task Parallelism Implementations:**

1. **Merge Sort** (in operations.c)
   After the block sorting phase, merge sort is used to merge the array segments. The merge tasks are structured by step levels, and at any given step, only threads satisfying the condition `tid % (2 * step) == 0` are assigned a task to merge their array segment with a sibling array segment. Threads that do not satisfy the condition on the other hand simply exit the loop.

## Code highlights :

1.To do dynamic data generation we had to fill in 4mb of data in the heap buffer malloc in RAM , fill it with random int and flush it in blocks to disk using fwrite()

Generate data file

```
        size_t w = fwrite(buf, sizeof(int32_t), to_write, f);
        if (w != to_write) {
            perror("fwrite failed");
            free(buf);
            fclose(f);
            return 1;
        }
        written += w;
    }

    free(buf);
    fclose(f);
    printf("Successfully generated 1GB test file: %s\n", out_name);
    return 0;
}
```

2.client child downloader socket

Client file

```
        // Read payload directly into shared memory offset
        uint64_t chunk_size = (file_size + N - 1) / N;
        uint64_t offset = (uint64_t)(seq - 1) * chunk_size;

        if (safe_recv(conn_fd, (char *)reassembly_buffer + offset, payload_size) != (ssize_t)payload_size) {
            fprintf(stderr, "[Client Child %u] Receive payload failed\n", seq);
            close(conn_fd);
            exit(1);
        }
```

## Individual Contribution Matrix

| Student | Modules/Functions Owned | Testing/Validation Role | Contribution files |
|---|---|---|---|
| Alhalah Kutaiba | Client child process, socket handeling reassembly buffer | Chunk integrity, MD5 verification | Client.c , generate_data.c , common.h |
| Muhammad Ammar bin Mohamad Idham | Server chunk division, fork management, file I/O | Socket stress test, error handling | server.c, verify.c |
| Muhammad Nafiz Bin Mohd Fauzi | Min/Max parallel reduction, mutex/condvar coordination | Timing analysis, thread count validation | operations.c, page_simulator.c |

| | | | |
|---|---|---|---|
| Muaz Wazif Bin Abd Manab | Parallel sort implementation, task scheduling, logging | Output correctness, sorted verification | operations.c, run_test.sh |

## Testing & Validation

### Socket stress test

Single threaded performance:

```
~/school/os/OS assignment
❯ ./operations -t 1 -f reassembled.dat
[Worker Thread 0] TID: 139927710848704 running on CPU Core: 0 (Processing segment
 [0 to 268435455])
[Worker Thread 0] Starting block sort of size 268435456 on CPU Core: 0...

~/school/os/OS assignment 55s
❯ cat execution_log.txt
[PART1] CHUNKS=4 | PROCS=4 | SYNC_USED=mutex,sem,condvar
[PART2] THREADS=8 | DATA_PARALLEL=min,max | TASK_PARALLEL=sort
[PART2] TIME_MS=12307 | SORT_ALGO=parallel_merge_sort
[STATUS] SUCCESS
[PART2] THREADS=1 | DATA_PARALLEL=min,max | TASK_PARALLEL=sort
[PART2] TIME_MS=38052 | SORT_ALGO=parallel_merge_sort
[STATUS] SUCCESS
```

Multithreaded performance:

```
~/school/os/OS assignment
❯ ./operations -t 8 -f reassembled.dat
[Worker Thread 1] TID: 140324145972928 running on CPU Core: 1 (Processing segment [33554432 to 67108863])
[Worker Thread 3] TID: 140324112410304 running on CPU Core: 3 (Processing segment [100663296 to 134217727])
[Worker Thread 4] TID: 140324095628992 running on CPU Core: 4 (Processing segment [134217728 to 167772159])
[Worker Thread 0] TID: 140324162754240 running on CPU Core: 0 (Processing segment [0 to 33554431])
[Worker Thread 5] TID: 140324078847680 running on CPU Core: 5 (Processing segment [167772160 to 201326591])
[Worker Thread 6] TID: 140324062066368 running on CPU Core: 6 (Processing segment [201326592 to 234881023])
[Worker Thread 7] TID: 140323963532992 running on CPU Core: 7 (Processing segment [234881024 to 268435455])
[Worker Thread 2] TID: 140324129191616 running on CPU Core: 2 (Processing segment [67108864 to 100663295])
[Worker Thread 3] Starting block sort of size 33554432 on CPU Core: 3...
[Worker Thread 5] Starting block sort of size 33554432 on CPU Core: 5...
[Worker Thread 7] Starting block sort of size 33554432 on CPU Core: 7...
[Worker Thread 4] Starting block sort of size 33554432 on CPU Core: 4...
[Worker Thread 2] Starting block sort of size 33554432 on CPU Core: 2...
[Worker Thread 6] Starting block sort of size 33554432 on CPU Core: 6...
[Worker Thread 0] Starting block sort of size 33554432 on CPU Core: 0...
[Worker Thread 4] Merging with Sibling 5 on CPU Core: 4 (Range: [134217728 to 167772159] and [167772160 to 201326591])
[Worker Thread 1] Starting block sort of size 33554432 on CPU Core: 1...
[Worker Thread 2] Merging with Sibling 3 on CPU Core: 2 (Range: [67108864 to 100663295] and [100663296 to 134217727])
[Worker Thread 6] Merging with Sibling 7 on CPU Core: 6 (Range: [201326592 to 234881023] and [234881024 to 268435455])
[Worker Thread 4] Merging with Sibling 6 on CPU Core: 4 (Range: [134217728 to 201326591] and [201326592 to 268435455])
[Worker Thread 0] Merging with Sibling 1 on CPU Core: 0 (Range: [0 to 33554431] and [33554432 to 67108863])
[Worker Thread 0] Merging with Sibling 2 on CPU Core: 0 (Range: [0 to 67108863] and [67108864 to 134217727])
[Worker Thread 0] Merging with Sibling 4 on CPU Core: 0 (Range: [0 to 134217727] and [134217728 to 268435455])
```

```
~/school/os/OS assignment 14s
❯ cat execution_log.txt
[PART1] CHUNKS=4 | PROCS=4 | SYNC_USED=mutex,sem,condvar
[PART2] THREADS=8 | DATA_PARALLEL=min,max | TASK_PARALLEL=sort
[PART2] TIME_MS=12307 | SORT_ALGO=parallel_merge_sort
[STATUS] SUCCESS
[PART2] THREADS=1 | DATA_PARALLEL=min,max | TASK_PARALLEL=sort
[PART2] TIME_MS=38052 | SORT_ALGO=parallel_merge_sort
[STATUS] SUCCESS
[PART2] THREADS=8 | DATA_PARALLEL=min,max | TASK_PARALLEL=sort
[PART2] TIME_MS=12172 | SORT_ALGO=parallel_merge_sort
[STATUS] SUCCESS
```

### Error handling

```
~/school/os/OS assignment
❯ ./client -p 4 -t 8
Connection failed: Connection refused

~/school/os/OS assignment
❯ ./server non_existent_file.dat
Failed to open input file: No such file or directory

~/school/os/OS assignment
❯ ./client -p 0
Number of chunks (N) must be greater than 0.

~/school/os/OS assignment
❯ ./operations -t 0 -f reassembled.dat
Number of threads (T) must be greater than 0.
```

### Socket stress test

Connection closing was temporarily hardcoded into client.c for the purposes of this test.

```
~/school/os/OS assignment
❯ ./server original.dat
[Server Child 2] PID: 107543 executing on CPU Core: 1 (Serving chunk 2/4)
[Server Child 1] PID: 107541 executing on CPU Core: 0 (Serving chunk 1/4)
[Server Child 4] PID: 107547 executing on CPU Core: 3 (Serving chunk 4/4)
[Server Child 3] PID: 107545 executing on CPU Core: 2 (Serving chunk 3/4)
Child process 1 failed with status 0

~/school/os/OS assignment
❯
```

```
~/school/os/OS assignment
❯ ./client -p 4
[Client Child 1] PID: 107542 executing on CPU Core: 0 (Downloading chunk 1/4)
[Client Child 2] PID: 107544 executing on CPU Core: 1 (Downloading chunk 2/4)
[Client Child 3] PID: 107546 executing on CPU Core: 2 (Downloading chunk 3/4)
[Client Child 4] PID: 107548 executing on CPU Core: 3 (Downloading chunk 4/4)
[Client Child 2] Intentionally closing connection to stress test server!
Client child 2 failed
~/school/os/OS assignment
❯
```

## Challenges & Solutions

1. Making the threads work properly

   Problem: Our sorting program created a new thread for every single data split. When testing the large 1GB file, the computer ran out of available threads. This caused our program crash.

   Solution: We created a thread budget tracking counter inside src/operations.c. The program only builds new thread if it has extra slots left in the budget. If the budget runs out, it automatically switches to a normal sequential sort to save resources.

2. Race conditions

   Problem: Multiple threads tried to update global minimum and maximum variables at the exact same time, which messed up our final calculations. Also, parentm sorting functions tried to merge data before child threads finished writing to memory

   Solution: We proteced our shared data inside src/operations.c using POSIX mutexes and condition variables.

   - For Min/Max: Threads track numbers using their own local variables. They only use a mutex lock (pthread_mutex_lock) at the very end to update the main record safely.

   - For Sorting: We forced parent functions to wait (pthread_cond_wait) until child worker threads send a clear signal (pthread_cond_broadcast) proving they are completely done.

## References


1. Lecture 3 (Processes)
2. Lecture 4 (Threads and Concurrency)
3. Lecture 6 (Synchronisation)
4. Lecture 7(deadlocks)
5. Lecture 9 and 10 (virtual memory)
6. Lectures 11 and 12 (file system)